

Name: Kashinath Sangram Hale

PRN no.: 202501040055

Division: CS-1 Batch:

C13 Course: EDS

Essential of Data Science Laboratory

Practical 2:

2.1.1 List Operations:

CODETANTRA

Home Learn Anywhere

202501040055@mitaoe.ac.in Support Logout

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.

• The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

listOps.py

```
1 lis=[]
2 while True:
3     print("1. Add")
4     print("2. Remove")
5     print("3. Display")
6     print("4. Quit")
7     choice=int(input("Enter choice: "))
8     if (choice == 1):
9         a1=int(input("Integer: "))
10        lis.append(a1)
11        print("List after adding:",lis)
12    elif choice==2:
13        if len(lis)==0:
14            print("List is empty")
15        else:
16            a2=int(input("Integer: "))
17            if a2 in lis:
18                lis.remove(a2)
19                print("List after removing:",lis)
20            else:
21                print("Element not found")
22    elif choice==3:
23        if len(lis)==0:
24            print("List is empty")
25        else:
26            print(lis)
27    elif choice==4:
28        break
29    else:
30        print("Invalid choice")
```

Terminal Test cases

< Prev Reset Submit Next >

CODETANTRA

HomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

2.1.1. List operations31.28

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

Average time0.180 s179.67 msMaximum time0.235 s235.00 ms

2 out of 2 shown test case(s) passed1 out of 1 hidden test case(s) passed

Test case 1146 ms

Expected outputActual output

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 11

List after adding: [11]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 11

List after adding: [11]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 25

List after adding: [11, 25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 11

List after adding: [11]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 11

List after adding: [11]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 25

List after adding: [11, 25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

< PrevResetSubmitNext >

2.1.2 Dictionary Operations:

CODETANTRA

HomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

2.1.2. Dictionary Operations01.24

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

DictOper...

```
1 # Initial dictionary with 10 predefined records
2 student = {
3     1: "Amit",
4     2: "Riya",
5     3: "Kiran",
6     4: "Neha",
7     5: "Arjun",
8     6: "Pooja",
9     7: "Rahul",
10    8: "Sneha",
11    9: "Vikram",
12    10: "Anjali"
13 }
14 print("Original Dictionary:", student)
15 n1 = int(input())
16 student[n1] = input()
17 print("After Insertion:", student)
18
19 n2 = int(input())
20 student[n2] = input()
21 print("After Update:", student)
22
23 n3 = int(input())
24 if n3 in student:
25     del student[n3]
26 print("After Deletion:", student)
27 print("Traversing Dictionary:")
28 for keys, values in student.items():
29     print(str(keys) + " : " + str(values))
```

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRA

HomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

2.1.2. Dictionary Operations

01:24

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.

2. Update – Update the value of an existing key using user input.

3. Deletion – Delete a specified key from the dictionary using user input.

4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.

2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.

3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary unchanged.

4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

All operations must be performed using dictionary methods.

Perform deletion only if possible; leave the dictionary unchanged.

Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

+

Explorer

Average time0.051 s51.25 ms

Maximum time0.061 s61.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 161 ms

Debug

Expected output

Original-Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

11

Suresh

After-Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

3

Karthik

After-Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

5

After-Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing-Dictionary:

1: Amit

2: Riya

3: Karthik

4: Neha

6: Pooja

Actual output

Original-Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

11

Suresh

After-Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

3

Karthik

After-Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

5

After-Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing-Dictionary:

1: Amit

2: Riya

3: Karthik

4: Neha

6: Pooja

< Prev

Reset

Submit

Next >

2.2.1 Linear Search Technique:

CODETANTRA

HomeLearn Anywhere

202501040055@mitaoe.ac.inSupportLogout

2.2.1. Linear search Technique

13:13

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

The first line of input contains the array of integers which are separated by space

The last line of input contains the key element to be searched

Output format:

If the element is found, print the index. If the element found multiple times, print the starting index

If the element is not found, print **Not found**.

Sample Test Case:

Input:

1 2 3 4 3 5 6

3

Output:

2

Sample Test Cases

+

Explorer

CTP1709...

Submit

1n = list(map(int,input().split(" ")))

2b = int(input())

3index = 0

4check = True

5for i in n:

6 if i == b:

7 print(index)

8 check = False

9 break

10 index+=1

11

12if check:

13 print("Not found")

Average time0.019 s19.00 ms

Maximum time0.024 s24.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 124 ms

Debug

Expected output

1 2 3 4 3 5 6

3

2

Actual output

1 2 3 4 3 5 6

3

2

Test case 220 ms

Debug

Expected output

Actual output

Terminal

Test cases

< Prev

Reset

Submit

Next >

2.2.2 Captain of the team:

CODETANTRA

Home Learn Anywhere

202501040055@mitaoe.ac.in Support Logout

2.2.2. Captain of the Team

14:32

AA

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

captainof...

Submit

```
1 n = list(map(int,input().split(" ")))
2 b=0
3 for i in n:
4     if b<i:
5         b=i
6 print(b)
```

Average time
0.016 s
15.67 ms

Maximum time
0.018 s
18.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 18 ms Debug

Expected output
171 169 185 156 174 191 186 190 187 172 160
191

Actual output
171 169 185 156 174 191 186 190 187 172 160
191

Terminal

Test cases

< Prev

Reset

Submit

Next >

